

Backend Take-Home Assignment

Custom CDC Pipeline: PostgreSQL → MongoDB

1. Background & Problem Statement

You are required to set up your own PostgreSQL instance, populate it with data, and build a custom Change Data Capture (CDC) pipeline that continuously streams row-level changes from PostgreSQL into a MongoDB collection, keeping the destination as fresh as possible.

The pipeline must be resilient under high write load. Staleness is acceptable within reason — data can be 30 seconds, 1 minute, or even 5 minutes behind — but the pipeline must never fall over, lose events, or require a full re-sync under normal operating conditions. How you achieve this is entirely up to you.

2. Objective

Build a working CDC pipeline that:

- Captures row-level changes (INSERT, UPDATE, DELETE) from one or more PostgreSQL source tables.
- Replicates those changes faithfully into MongoDB, keeping the destination as fresh as possible.
- Handles high write load gracefully — it may lag, but it must not drop events or crash.
- Propagates schema changes in PostgreSQL to MongoDB in a controlled way (see Section 4.4).
- Is runnable locally via Docker Compose with a single command.
- Is accompanied by a correctness test suite and a written design document.

3. Source Schema & Data

3.1 Tables

You are free to design your own schema. Use any publicly available dataset or invent one. The schema must satisfy the following constraints:

- At least 2 tables with a foreign-key relationship between them.

- Each table must have at least 6 columns of mixed types: integer or UUID primary key, text fields, a numeric or monetary column, at least one timestamp, and a boolean or enum.
- Realistic column naming (not col1, col2).

Suggested schemas (pick one or design your own):

Domain	Tables (example)	Notes
E-commerce	orders, order_items	order status enum, amounts, timestamps
Social / Content	users, posts	follower counts, like counts, visibility flag
Ride-hailing	drivers, trips	lat/lng as numeric, status enum, fare
Finance	accounts, transactions	balance, txn type, debit/credit amounts

3.2 Data Volume

Seed each table with a realistic volume of rows before the pipeline starts. We expect:

- Minimum 500,000 rows in the primary table at snapshot time.
- Child / related table proportional (e.g., 2–5 rows per parent row).

Use a seed script (Python, SQL, or any tool) that is included in the repo and documented in the README. The reviewer must be able to reproduce the seed with one command.

4. Pipeline Requirements

4.1 Initial Snapshot

- On first run, perform a consistent snapshot of all source tables before switching to streaming mode.
- No changes made during the snapshot window must be lost — the transition from snapshot to stream must be seamless.
- Snapshot progress should be observable (logs, counter, or a simple status output).

4.2 Streaming Replication

- All three DML operations must be replicated: INSERT, UPDATE, DELETE.

- UPDATES should be applied as partial updates on the MongoDB document, not full replacement, where possible.
- DELETES must either remove the document or apply a soft-delete (deleted_at field). Be consistent and document your choice.
- The pipeline must track its replication offset persistently so a restart does not cause re-processing of already-applied events.

4.3 Delivery Guarantee

- The pipeline must provide at-least-once delivery.
- Explain in your design doc how duplicate events are handled on the MongoDB write side (idempotency strategy).

4.4 Schema Change Propagation

Schema changes in PostgreSQL (e.g., ALTER TABLE ADD COLUMN, ALTER TABLE DROP COLUMN) must be handled explicitly. The expected behaviour is:

- The pipeline must not crash silently on a schema change.
- New columns added to a PostgreSQL table should not automatically appear in MongoDB unless the operator explicitly triggers a schema sync.
- Implement a schema sync mechanism — a command or flag that, when invoked, aligns the MongoDB collection schema with the current PostgreSQL table definition and backfills missing fields.
- Until a schema sync is triggered, the pipeline should continue replicating existing columns and skip or null-fill the new ones, with a clear log warning.

Think of this like Airbyte's schema change behaviour: the system detects a drift, surfaces it, and waits for an explicit operator action before propagating it.

5. Load Simulation & Resilience

This is the core of what we are evaluating. The pipeline must be stress-tested as part of the submission.

5.1 Load Simulation Script

Write a load simulation script that runs the following workload against the PostgreSQL source continuously:

Operation	Target Rate	Notes
INSERT	3,000–5,000 rows/sec	Across both tables, maintaining FK integrity
UPDATE	1,000–2,000 rows/sec	Random rows; update 1–3 columns per row
DELETE	200–500 rows/sec	Random rows; avoid deleting rows with live FK deps

The script must be runnable independently (e.g., `python load_test.py --duration 120`) and must print a summary at the end: total ops, ops/sec, any errors.

5.2 What We Expect You to Measure

While the load script runs, observe and record:

- Replication lag: how far behind is MongoDB relative to PostgreSQL? Measure in seconds using LSN offset or a timestamp column.
- Does the pipeline recover after the load burst ends? How quickly does lag drain?
- Where is the bottleneck — the reader, the transport, or the writer?
- Does the pipeline fail hard (crash, error) or soft (lag grows but self-heals) under peak load?
- What is the approximate maximum sustainable throughput before lag grows unboundedly?

Include your observations and numbers in the design document. Screenshots or log excerpts are acceptable as evidence.

5.3 Tunable Parameters

Document every meaningful configuration knob in your pipeline and the effect of tuning it. This must go beyond listing defaults — explain what you actually changed during load testing and why.

At minimum, cover parameters in these categories:

- Source read settings (batch size, polling interval, WAL buffer limits if applicable)
- Transport / queue settings if you introduced one (buffer sizes, concurrency, retry limits)
- Destination write settings (batch size, write concern, connection pool size)
- Any back-pressure or flow control mechanism you implemented or configured

For each parameter, state: the default value, what you set it to, and the observed effect on lag or throughput.

6. Correctness Test Suite

Include an automated test suite that verifies the pipeline is replicating correctly. Tests should be runnable against a live pipeline (docker compose up) with a single command: e.g., `python tests/run_correctness.py` or `pytest tests/`.

6.1 Required Test Cases

#	Test Name	What It Verifies
T1	Insert propagation	Insert N rows into PG; assert all N rows appear in MongoDB within a configurable timeout.
T2	Update propagation	Update a specific field on K rows; assert MongoDB documents reflect the updated value.
T3	Delete propagation	Delete M rows from PG; assert they are absent (or soft-deleted) in MongoDB.
T4	Bulk insert consistency	Insert 10,000 rows in a single transaction; assert final count in MongoDB matches PG exactly.
T5	Row-level field parity	Pick 100 random rows; assert every replicated field matches byte-for-byte between PG and Mongo.
T6	Restart idempotency	Kill and restart the pipeline mid-stream; assert no duplicate or missing documents after recovery.
T7	Schema change — new column	ALTER TABLE ADD COLUMN in PG; assert pipeline does not crash; assert new column does not appear in Mongo until schema sync is triggered.
T8	Schema sync	Trigger schema sync; assert the new column now exists on all existing Mongo documents (null-filled where absent).
T9	Replication lag bound	Under 2× normal load, assert that replication lag never exceeds your stated SLO (document the SLO value).

T1 0	Snapshot completeness	After initial snapshot, assert row count in MongoDB equals row count in PG for each table.
---------	-----------------------	--

Tests T7 and T8 are bonus if schema change propagation is not fully implemented; T1–T6 and T10 are required.

6.2 Test Output

- Each test must print PASS / FAIL with a short reason.
- Include a summary line at the end: X/10 tests passed.
- Tests must be deterministic — running them twice in a row must yield the same result.

7. Design Document (Required)

Include a DESIGN.md in your repo. It must cover the following sections:

7.1 Architecture Diagram

A diagram (ASCII or image) showing every component — Postgres, your capture mechanism, any intermediate layer, the MongoDB writer. Label each arrow with the data format in flight.

7.2 How the Pipeline Works

- Step-by-step data flow: from a committed transaction in PG to a document written in MongoDB.
- How the replication offset is tracked and persisted across restarts.
- How the snapshot-to-stream handoff works without losing events.
- Why you chose your capture approach over alternatives.

7.3 Staleness & Resilience Analysis

- What is your measured replication lag at baseline load? At peak load?
- What is the maximum sustainable throughput before lag grows unboundedly?
- Does the pipeline fail hard or soft under overload? What is the recovery behaviour?
- What would you change to handle 10× the current load?

7.4 Tuning Guide

- Full table of parameters: name, default, your value, observed effect.

- Note any parameters that must be co-tuned.

7.5 Failure Modes

Address at least three of the following:

- PostgreSQL restart / WAL slot invalidation
- MongoDB write failure (timeout, duplicate key)
- Pipeline crash mid-batch — which events are replayed?
- WAL slot bloat consuming disk (replication slot age)
- Schema change not yet synced — what happens to rows inserted after the ALTER?

7.6 Trade-offs & What You Would Do Differently

- Why this approach over alternatives?
- Known gaps or shortcuts taken due to time constraints.
- What a production-grade version of this pipeline would look like.

8. Deliverables Checklist

#	Item	Required?
1	GitHub repo with working code and full commit history	Required
2	docker-compose.yml — docker compose up brings up PG, MongoDB, and the pipeline	Required
3	Seed script — one command populates source tables with 500K+ rows	Required
4	Load simulation script with summary output	Required
5	Correctness test suite covering T1–T6 and T10 at minimum	Required
6	README.md: setup, how to run seed, load test, and correctness tests	Required
7	DESIGN.md covering all sections in Section 7	Required
8	Schema change handling (T7 + T8 tests)	Bonus
9	Replication lag metric exposed (Prometheus, logs, or a simple printed dashboard)	Bonus
10	Multi-table replication with FK-aware ordering	Bonus

9. Evaluation Criteria

Dimension	What We Look For	Weight
Pipeline correctness	All DML ops replicated; snapshot consistent; no loss on restart	25%
Resilience under load	Evidence-based bottleneck ID; lag behaviour at peak; recovery	25%
Design quality	Clear architecture; honest trade-off analysis; failure modes covered	20%
Test suite quality	Tests are deterministic, meaningful, and actually catch bugs	15%
Code & ops quality	Readable, modular, containerised; README is self-sufficient	15%

10. Ground Rules

- You may use any language and any open-source libraries.
- The choice of capture mechanism, transport layer, and write strategy is entirely yours. There is no prescribed approach — we want to see your reasoning, not a specific tool.
- If you use a managed connector or off-the-shelf CDC tool, you must be able to explain every configuration option in your design doc. Black-box usage will score poorly on the resilience and tuning dimensions.
- Do not build a UI or dashboard — this is a backend engineering assignment.
- If you run out of time, document what you would have done next in DESIGN.md. An honest gap analysis is valued over a rushed implementation.
- Your submission will be reviewed in a 45-minute technical interview. Be prepared to live-explain your design, modify tuning parameters on the fly, and discuss what happens under 10× load.

11. Questions

If anything is unclear, reach out before you start. We would rather answer a question upfront than evaluate a submission that misunderstood the brief.

Good luck.